SIMATS ENGINEERING

CSA02 - C Programming

ASSIGNMENT 5

CO5 | 50 marks

Modular, Memory-Aware and Persistent Program Engineering

Functions, recursion, pointers, files and measured engineering decisions

Complete solutions for Tasks A1-A5 and B1-B4

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository:
Probir127/CSA02-1911260319-Assignment5

Scope, assumptions and execution environment

PROBIR SAHA SHOHOM | Register Number: 1911260319.

Repository: https://github.com/Probir127/CSA02-1911260319-Assignment5

This report covers all five Part A tasks and four Part B tasks, including the three supplied code-completion programs. All executable solutions are C11; the reproduction runner is Bash. The program uses synthetic data and a declared demonstration eligibility rule: recorded age 18 through 65 inclusive and at least 90 days since the last recorded donation. This is a programming test rule, not a clinical policy. The assignment does not supply a medical eligibility rule.

Execution date: 11 November 2026. Compiler invoked as gcc: Apple Clang 21.0.0 (clang-2100.3.34.2), target arm64-apple-darwin27.0.0. Standard build: -std=c11 -Wall -Wextra -Wpedantic -Werror -O0 -g. Exact compile/run commands are in tools/run_all.sh and README.md. All intentional unsafe examples run in separate child processes. Predicted counts were recorded before execution in PREDICTIONS.md.

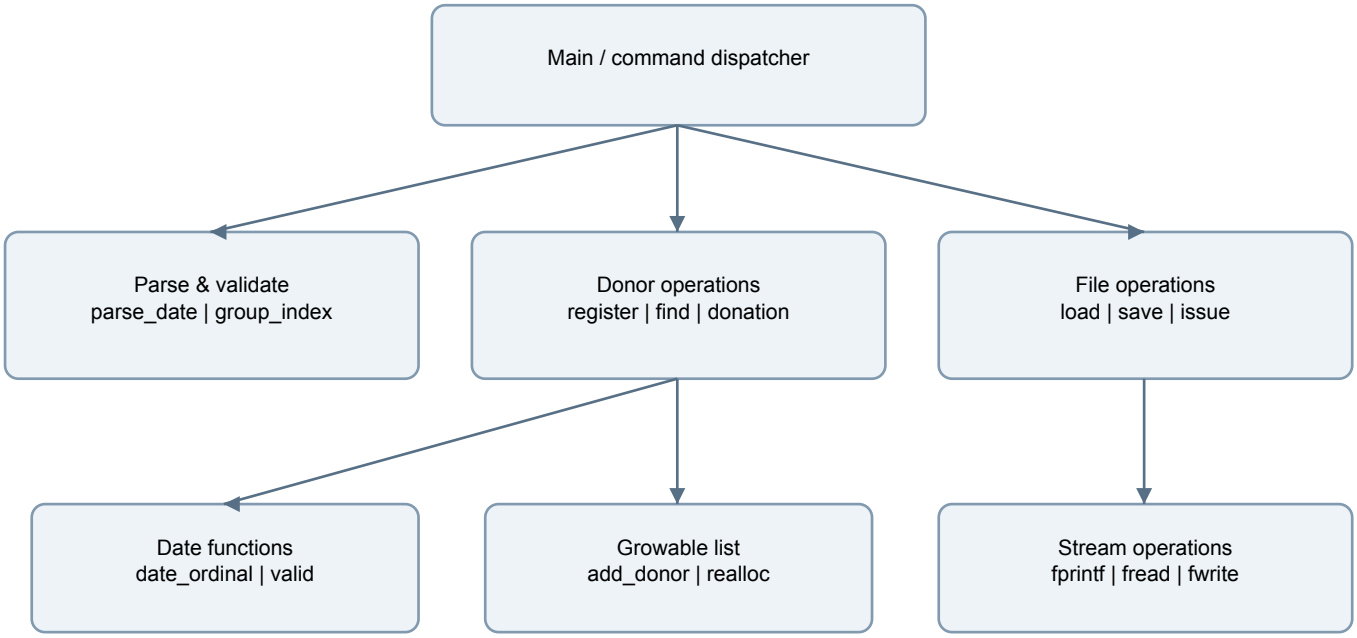
Pseudocode uses ALGORITHM, INPUT, OUTPUT, BEGIN/END and explicit control blocks. The separately named SIMATS pseudocode style guide was not supplied; exact typography against that separate guide remains unverified. C listings are included in addition to the Part A pseudocode, not as its replacement.

The output PNGs are labelled renderings of the accompanying captured program logs. They are not personal terminal screenshots. The brief's individual execution, personal screenshot and faculty-sharing requirements remain actions for the student; this report does not claim those actions were performed by the student. Commit dates remain their actual dates.

Task A1 - Functional decomposition and interfaces (7 marks)

A1.1 - Responsibilities and structure chart

The program separates domain rules (`bank.c`), Gregorian dates (`date.c`), persistence (`storage.c`) and executable drivers. `register_donor` validates a record, rejects duplicate IDs and inserts into ID order. `record_donation` checks the demonstration eligibility rule, adds one stock unit and updates the date. `update_stock` rejects insufficient stock and integer overflow. The CLI provides find, issue and stock commands; the A1 driver exercises registration and donation through the same module API.




```
ALGORITHM Register_Donor
INPUT: mutable list L, read-only donor D
OUTPUT: success or failure
BEGIN
    IF D is invalid THEN RETURN failure END IF
    position <- LOWER_BOUND(L, D.id)
    IF position holds D.id THEN RETURN failure END IF
    copy <- D
    old_size <- L.size
    IF Add_Donor(L, copy) fails THEN RETURN failure END IF
    SHIFT records [position, old_size) one place right
    L[position] <- copy
    RETURN success
END

ALGORITHM Record_Donation
INPUT: mutable donor list L, mutable stock S, ID, date T
OUTPUT: success or failure
BEGIN
    FIND D with the requested ID
    IF no D OR not Eligible_On(D, T) THEN RETURN failure END IF
    IF Update_Stock(S, D.group, +1) fails THEN RETURN failure END IF
    D.last_donation <- T
    RETURN success
END
```

A1.2 - Prototypes and parameter decisions

`common/bank.h` and `common/date.h` contain the public prototypes below. A C pointer itself is passed by value; the pointed-to object can be changed through a non-const pointer. Array parameters decay to pointers. Small scalars and the 12-byte Date are copied by value. Donor records and lists are passed by pointer to avoid copying their contents or losing ownership changes.

```
int group_index(const char *group);
unsigned digital_root(unsigned long long value);
bool donor_valid(const Donor *donor);
void list_init(DonorList *list,Allocator *allocator);
void list_free(DonorList *list);
int add_donor(DonorList *list,const Donor *donor);
int register_donor(DonorList *list,const Donor *donor);
bool eligible_on(const Donor *donor,Date on);
Eligible find_eligible(const DonorList *list,const char *group,Date on);
void find_eligible_out(const DonorList *list,const char *group,Date on,size_t *count,int
    *youngest_id);
int *lowest_stock(int stock[],size_t count);
int update_stock(int stock[GROUP_COUNT],int group,int delta);
int record_donation(DonorList *list,int stock[GROUP_COUNT],int id,Date on);
bool parse_date(const char *text,Date *out);
bool parse_unsigned(const char *text,unsigned maximum,unsigned *out);
Date date_from_ordinal(int ordinal);
Date date_add(Date date,int days);
int save_donors(const char *path,const DonorList *list);
int load_donors(const char *path,DonorList *list);
int save_binary_donors(const char *path,const DonorList *list);
int load_binary_donors(const char *path,DonorList *list);
int save_stock(const char *path,const int stock[GROUP_COUNT]);
int load_stock(const char *path,int stock[GROUP_COUNT]);
int issue_file(const char *path,int group,unsigned units);
int update_binary_donor(const char *path,size_t index,const Donor *donor);
bool same_donor(const Donor *a,const Donor *b);
Donor make_donor(unsigned sequence,Date today);
bool date_valid(Date d);
int date_ordinal(Date d);
```

Functions / parameter names	Mode and justification
group_index(group), parse_date(text,out), parse_unsigned(text,maximum,out)	Strings are const char pointers; parser output pointers are mutable. Maximum is copied. Parsing failure leaves output unchanged.
digital_root(value), date_valid(d), date_ordinal(d), date_from_ordinal(ordinal), date_add(date,days)	Scalar or small Date values; no caller mutation. Ordinal conversion/date addition require an in-range date/result.
donor_valid(donor), same_donor(a,b)	const Donor pointers: inspect complete records without modifying or copying them.
list_init(list,allocator), list_free(list)	Mutable list pointer updates ownership. Allocator pointer is mutable for counters; it must outlive the list. Free resets data/size/capacity.
add_donor(list,donor), register_donor(list,donor)	Mutable list, const donor. Copy the donor before realloc so aliases into the old allocation are safe. add_donor appends; register_donor preserves sorted IDs.
eligible_on(donor,on), find_eligible(list,group,on)	Read-only donor/list/group pointers and a by-value date; no data modification.
find_eligible_out(list,group,on,count,youngest_id)	Same read-only inputs, plus mutable output pointers. This implementation accepts NULL to omit an output.
lowest_stock(stock,count), update_stock(stock,group,delta)	Mutable stock pointer enables updating the selected element. Length/group/delta are copied. Lowest returns NULL for an empty array.
record_donation(list,stock,id,on)	Mutable list and stock pointers change two in-memory objects; ID and Date copied.
read_line(file,buffer,capacity)	FILE pointer changes stream position; mutable buffer receives text; capacity copied and bounds checked.
save_donors(path,list), save_binary_donors(path,list), save_stock(path,stock)	Path and list/stock are const: save observes caller data but writes an external file.
load_donors(path,list), load_binary_donors(path,list), load_stock(path,stock)	const path plus mutable output. Donor loads stage into a temporary list and preserve the old list on failure.

Functions / parameter names	Mode and justification
issue_file(path,group,units), update_binary_donor(path,index,donor)	const path/donor, by-value selector and quantity; modify an external file, not the input record.
make_donor(sequence,today)	By-value fixture inputs; return an independently owned Donor structure by value.

Private storage helpers have prototypes `static int invalid(const char *context)` and `static bool parse_donor(char *line, Donor *d)`. The first reads a message; the second splits a mutable line and fills an output record. The command driver uses `static int usage(void)` and `int main(int argc, char **argv)`; argv strings are only read. Experiment-only helpers are local to their C files, with full definitions in the source appendix.

A1.3 - Returning two results

```
ALGORITHM Find_Eligible_Struct
INPUT: read-only list L, group G, date T
OUTPUT: structure {count, youngest_id, youngest_age}
BEGIN
    result <- {0, 0, 0}
    FOR each donor D in L DO
        IF D.group = G AND Eligible_On(D, T) THEN
            IF result.count = 0 OR D.age < result.youngest_age
               OR (D.age = result.youngest_age AND D.id < result.youngest_id) THEN
                result.youngest_id <- D.id
                result.youngest_age <- D.age
            END IF
            result.count <- result.count + 1
        END IF
    END FOR
    RETURN result
END

ALGORITHM Find_Eligible_Pointers
INPUT: L, G, T, output addresses count_out and id_out
OUTPUT: values written through valid output addresses
BEGIN
    result <- Find_Eligible_Struct(L, G, T)
    IF count_out is not NULL THEN *count_out <- result.count END IF
    IF id_out is not NULL THEN *id_out <- result.youngest_id END IF
END
```

Both designs scan once: O(n) time and O(1) extra space. The small returned structure is easy to name and cannot suffer mismatched output addresses. Pointer outputs are useful when callers want to omit fields, but add NULL and aliasing decisions. Zero count and ID zero mean no match; ties use the smaller ID. The driver confirms both interfaces agree.

A global-variable alternative stores `DonorList donors`, `int stock[8]`, `count` and `youngest_id` at file scope and uses `find_eligible_global(group,date)`. The scan remains O(n) and the dataset remains O(n) storage; only a constant-size argument/result mechanism changes. Hidden dependencies increase coupling, prevent independent banks from being processed without resetting state, complicate concurrent calls and make tests order-dependent. Passing explicit pointers retains the same asymptotic cost and makes ownership and mutation visible.

Executed evidence: `TaskA1/demo.c`, `TaskA1/output.txt`; registration, duplicate rejection, eligibility today/next week, donation, repeated-donation rejection, issue rejection and text/binary reload all passed.

Task A2 - Recursive designs and traces (5 marks)

A2.1 - Digital root and halving power

For IDs, the final decimal digit equals the digital root of the preceding digits. For example, body 100000 has root 1, hence ID 1000001. The following designs assume a nonnegative integer and a nonnegative exponent. Power inputs must fit the selected integer representation; the execution tests use bounded values and base 1 for large exponents.

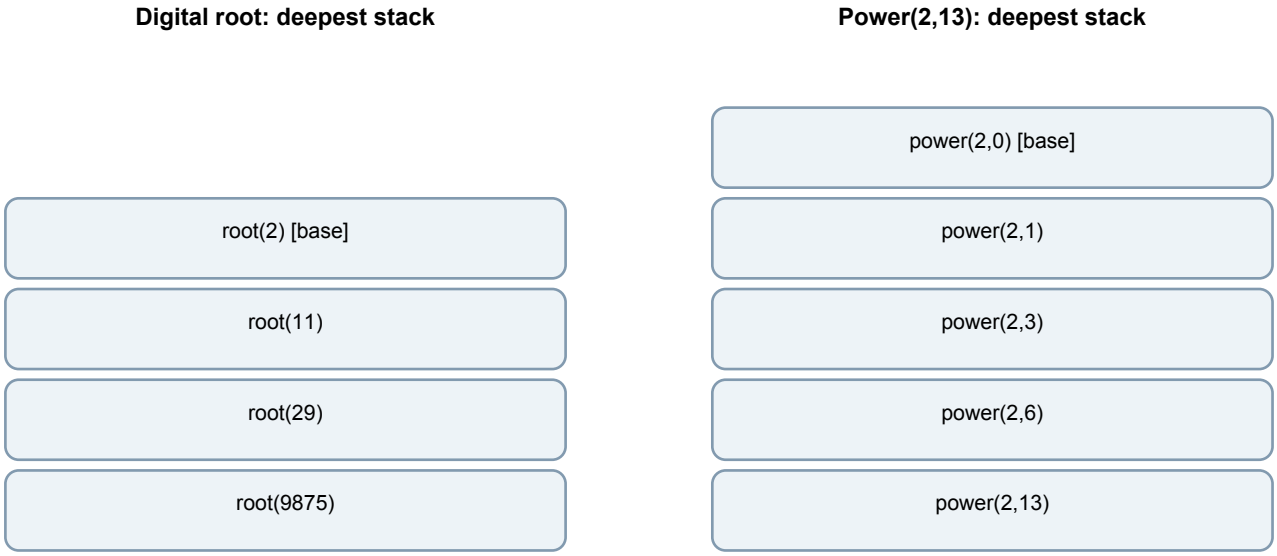
```
ALGORITHM Digital_Root
INPUT: nonnegative integer N
OUTPUT: one decimal digit
BEGIN
    IF N < 10 THEN RETURN N END IF
    sum <- 0
    WHILE N > 0 DO
        sum <- sum + (N MOD 10)
        N <- N DIV 10
    END WHILE
    RETURN Digital_Root(sum)
END

ALGORITHM Power_Fast
INPUT: base B, nonnegative exponent N
OUTPUT: B raised to N
BEGIN
    IF N = 0 THEN RETURN 1 END IF
    half <- Power_Fast(B, N DIV 2)
    IF N MOD 2 = 0 THEN RETURN half * half END IF
    RETURN half * half * B
END

ALGORITHM Power_Linear
INPUT: B, N
OUTPUT: B raised to N
BEGIN
    IF N = 0 THEN RETURN 1 END IF
    RETURN B * Power_Linear(B, N - 1)
END
```

The digital-root base case is a single digit, including zero. The recursive case first sums the digits. Power has base case N=0; halving computes the child exactly once. The convention 0^0=1 is explicit for this exercise.

A2.2 - Deepest stacks, return values and recurrences



For 9875: 9+8+7+5=29, 2+9=11, 1+1=2. Four calls remain live at the deepest point. Linear power for 2^13 has frames 13,12,...,1,0: 14 calls and return values 1,2,4,...,8192. Fast power has five frames for exponent 13.

Counting the initial and base-case calls: L(0)=1 and L(n)=1+L(n-1), so L(n)=n+1. F(0)=1 and F(n)=1+F(floor(n/2)), so F(n)=floor(log2(n))+2 for n>=1. Linear time/stack are O(n); fast time/stack are O(log n), under fixed-width, constant-time multiplication.

If D is the number of decimal digits, the first digital-root digit pass is O(D), the next has at most O(log D) digits, then smaller passes. Total digit-processing time is O(D). With iterative digit summation inside each call, stack depth is the number of repeated digit sums, much smaller than D (bounded by O(D); more tightly O(log-star N) over unbounded mathematical integers). For the finite unsigned-long-long domain, the depth is small. A version recursively processing each individual digit instead would have O(D) stack.

A2.3 - Risks and alternatives

Without a terminating base case, or with a recursive argument that does not approach it, calls exhaust the stack. C does not guarantee tail-call elimination. Use a loop for potentially unbounded linear recursion and use halving when the recurrence supports it. Stack growth is separate from numeric overflow: 2^10000 overflows long long even if its recursion fits.

```
ALGORITHM Digital_Root_Loop
INPUT: nonnegative N
OUTPUT: digital root
BEGIN
    WHILE N >= 10 DO
        sum <- 0
        WHILE N > 0 DO
            sum <- sum + N MOD 10
            N <- N DIV 10
        END WHILE
        N <- sum
    END WHILE
    RETURN N
END

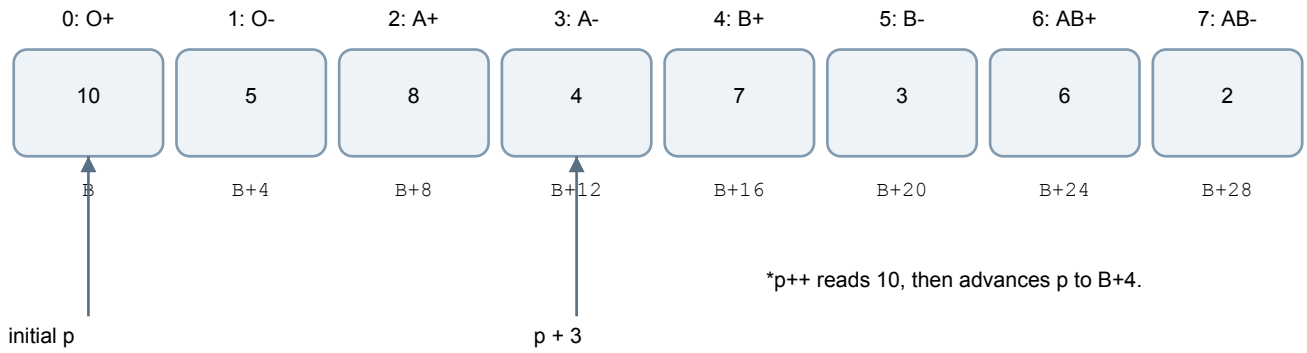
ALGORITHM Digital_Root_Formula
INPUT: nonnegative N
OUTPUT: digital root
BEGIN
    IF N = 0 THEN RETURN 0 END IF
    RETURN 1 + (N - 1) MOD 9
END
```

The loop uses O(D) digit-processing time and O(1) stack. The formula is O(1) time/space for machine integers; arbitrary-precision modulo is not necessarily O(1). The zero guard also prevents unsigned underflow at N-1. `TaskA2/design_check.c` confirms the trace values; B1 adds edge cases and measured calls.

Task A3 - Pointer operations and CLI (5 marks)

A3.1 - Pointer memory diagram

For `int stock[8]={10,5,8,4,7,3,6,2}; int *p=stock;`, measured sizeof(int)=4 on this system. B denotes the base address, so pointer arithmetic scales by four bytes.



`p+3` points at stock[3] without changing p; `*(p+3)` and `p[3]` both read 4. Postfix increment binds more tightly than `*`, so `*p++` means `*(p++)`: read the old pointed-to element and advance p by one int. It does not increment the stock value. These expressions are considered with the original p, except the explicitly shown post-increment. Pointer arithmetic must stay within the same array or its one-past position; one-past must not be dereferenced.

A3.2 - Returning a safe element pointer

```
ALGORITHM Lowest_Stock
INPUT: mutable array S, length N
OUTPUT: pointer to first minimum element, or NULL
BEGIN
    IF S is NULL OR N = 0 THEN RETURN NULL END IF
    best <- ADDRESS(S[0])
    FOR i <- 1 TO N - 1 DO
        IF S[i] < *best THEN best <- ADDRESS(S[i]) END IF
    END FOR
    RETURN best
END
```

The caller can use `int *p=lowest_stock(stock,8); if(p) *p+=3;`. The program selected index 7 and changed its value from 2 to 5. Returning `&local_minimum` would be invalid after the function returns because the local object's lifetime ends. Returning a pointer into the caller's still-live array is valid. A stock increment still needs a range/overflow check in a real update; the demonstrated 2+3 is bounded.

The index alternative returns `SIZE_MAX` for an empty array and otherwise the first minimum index. The caller checks `i != SIZE_MAX && i < count` before using `stock[i]`. Both alternatives require O(n) time and O(1) extra storage. After successful realloc, a saved element pointer is invalid even if the allocation appears unchanged. A saved index can be reapplied to the new base if the record remains at that index; insertions/deletions may still change its meaning.

A3.3 - Command-line contract

```
ALGORITHM Dispatch
INPUT: argc and argv
OUTPUT: printed result and process status
BEGIN
    IF argc < 2 THEN PRINT usage; RETURN 2 END IF
    IF command = "find" THEN
        REQUIRE argc = 5, valid group, valid YYYY-MM-DD
        LOAD donors; FIND eligible donors; PRINT count and youngest ID
    ELSE IF command = "issue" THEN
        REQUIRE argc = 5, valid group, integer units in [1, INT_MAX]
        UPDATE selected binary stock record
    ELSE IF command = "stock" THEN
        REQUIRE argc = 3; LOAD and PRINT all eight stock values
    ELSE PRINT usage; RETURN 2
    END IF
    ON file/data failure: PRINT perror message; RETURN 1
    FREE any owned donor allocation
    RETURN 0
END
```

Exact examples: `build/bloodbank find data/donors_1000.tsv O+ 2026-11-18`, `build/bloodbank issue build/readonly.bin A- 2`, `build/bloodbank stock data/stock.bin`. File paths and the reporting date are explicit so runs can be reproduced. Strict parsing rejects signs, trailing characters, overflow, invalid dates and unknown groups. Argument counts are checked before indexing later argv entries; shell quoting supports paths with spaces. B2 records the missing/extra/invalid argument cases.

Task A4 - Growable storage and ownership (4 marks)

A4.1 - Doubling with failure safety

The invariant is 0 <= size <= capacity <= 1,000,000. Only `list_free` owns cleanup. Appending does not require sorted IDs; registration and file loading enforce ID order. The richer Donor extends the supplied ID/group exercise with the fields needed by eligibility reporting, while retaining its same doubling/realloc algorithm.

```
ALGORITHM Add_Donor
INPUT: mutable list L, valid donor D
OUTPUT: success or failure
BEGIN
    IF D is invalid OR L.size = MAX_DONORS THEN RETURN failure END IF
    copy <- D
    IF L.size = L.capacity THEN
        IF L.capacity = 0 THEN new_capacity <- 2
        ELSE new_capacity <- MIN(2 * L.capacity, MAX_DONORS)
        END IF
        IF new_capacity > SIZE_MAX DIV SIZEOF(Donor) THEN
            RETURN failure
        END IF
        p <- REALLOC(L.data, new_capacity * SIZEOF(Donor))
        IF p is NULL THEN RETURN failure END IF
        L.data <- p
        L.capacity <- new_capacity
    END IF
    L.data[L.size] <- copy
    L.size <- L.size + 1
    RETURN success
END
```

The one-million cap makes integer doubling safe in the tested int representation; byte-size overflow is separately checked. Copying D before realloc handles D pointing inside the original array. On failure, realloc leaves the old allocation valid: size, capacity, pointer and records must remain unchanged. The injected-failure test confirms that invariant and then retries successfully.

A4.2 - Three memory errors

Error	Blood-bank example	Prevention
Leak	Assigning `list.data=realloc(list.data,bytes)` loses the only old pointer when allocation fails.	Use temporary p; adopt only on success. Free temporary lists on malformed input.
Dangling pointer	Cache `Donor *d=&list.data[0]`, grow the list, then use d after realloc invalidates it.	Retain an ID/index and resolve again; copy aliasing inputs before resize. Never return an automatic local's address.
Double free	Both a copied DonorList and its original believe they own the same data pointer.	Keep a single owner, transfer ownership explicitly, and reset after free. Do not shallow-copy owners for independent cleanup.

The implementation's allocator tracks live owned donor buffers. `list_free` is idempotent on the same reset list; it does not make two stale shallow copies safe.

Alternative: grow by ten

Replace doubling with `new_capacity=capacity+10`, retaining the temporary pointer and overflow checks. For n=10k, possible copied records sum to 10(0+1+...+k-1)=5k(k-1), hence O(n^2) overall. With doubling from 2, the old capacities form 2+4+...+C/2=C-2<2n, hence O(n) total copying and amortized O(1) append. A single growth can still be O(n).

Strategy	n	Final capacity	Growth calls	Potential records copied	Unused slots
Double	50	64	6	62	14

Strategy	n	Final capacity	Growth calls	Potential records copied	Unused slots
Add 10	50	50	5	100	0
Double	50,000	65,536	16	65,534	15,536
Add 10	50,000	50,000	5,000	124,975,000	0
Double	100,000	131,072	17	131,070	31,072
Add 10	100,000	100,000	10,000	499,950,000	0

These counts were produced by the growth simulation in `TaskA4/growth.c`; they are logical upper bounds if each realloc moves the buffer, not a measurement of physical copying. Add-ten wastes at most 9 slots after a successful append (10 immediately after growth before insertion); doubling wastes fewer than n slots after insertion (roughly half its capacity immediately after growth). Both use O(n) allocated space. Sorted registration also shifts records, so its O(n) insertion cost remains even though append allocation is amortized O(1).

Task A5 - Persistent text and binary formats (4 marks)

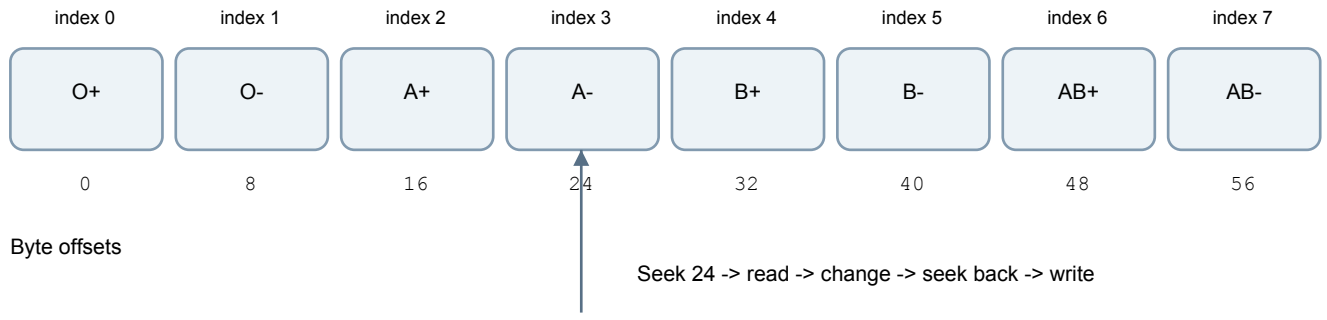
A5.1 - Storage design and round-trip rules

Donor text starts with the schema header `DONORS\t1`, followed by one tab-separated record per line: ID, name, group, age, phone, YYYY-MM-DD and hospital. `fprintf` makes records readable and easy to export. Names may contain spaces but not control characters/tabs; lengths, checksums, dates and IDs are validated. Loading demands strictly increasing IDs, rejects duplicates/truncation, and only replaces the active list after complete success.

Stock uses eight fixed records `struct {int group_code; int units;}` in the declared group order. Measured sizeof(Stock)=8; the file is 64 bytes. `fwrite` writes the array; `fread` validates all eight records and rejects extra bytes. Native struct files are suitable only for the same compiler/ABI in this exercise: integer widths, byte order and padding are not a portable external format.

```
ALGORITHM Save_Donors_Text
INPUT: path P, read-only list L
OUTPUT: success or failure
BEGIN
    OPEN P for text writing; IF failure THEN perror; RETURN failure END IF
    WRITE schema header using fprintf
    FOR each D in L DO
        WRITE D fields and newline using fprintf
        IF write fails THEN CLOSE file; RETURN failure END IF
    END FOR
    CHECK fclose result
    RETURN success only if all operations succeeded
END
```

A5.2 - One-record update




```
ALGORITHM Set_Stock_Units_In_Place
INPUT: open update stream F, record index I, nonnegative U
OUTPUT: success or failure
BEGIN
    IF I < 0 OR U < 0 THEN RETURN failure END IF
    REQUIRE I * SIZEOF(Stock) fits the seek offset type
    IF FSEEK(F, I * SIZEOF(Stock), START) fails THEN RETURN failure END IF
    IF FREAD(F, one Stock, S) does not read one record THEN
        RETURN failure
    END IF
    S.units <- U
    IF FSEEK(F, -SIZEOF(Stock), CURRENT) fails THEN RETURN failure END IF
    IF FWRITE(F, S, one record) fails THEN RETURN failure END IF
    RETURN success
END
```

Seeking before the write also satisfies the positioning requirement when switching an update stream from reading to writing. The caller checks flush/close for delayed errors. The production-style `issue_file` additionally checks group identity, nonnegative stock and sufficient units before reducing units. A5's test changes A- from 4 to 2 at byte offset 24 and verifies the other seven records.

A text change from 9 to 100 needs more characters and would overwrite the next field if edited directly. In the text alternative, open input and a temporary output, parse each record, replace the selected units, write every record, check

close, then rename the temporary over the original. This is O(n) per update with O(1) extra record memory when streaming, or O(n) RAM if the whole list is loaded as in B3. Fixed-record binary update takes O(1) record operations. Both formats use O(n) disk space; text size depends on decimal digits and separators, binary is n*sizeof(Stock). Text is not necessarily larger.

Normal close/reload persistence is implemented and tested. This exercise does not provide crash-atomic updates across both donor and stock files, concurrent hospital writes, or durable fsync/journaling. A real network needs transactions or a journal; fclose alone cannot guarantee no data loss during power failure. The text benchmark uses a temporary file and rename, but the basic save functions write directly to their selected paths.

Task B1 - Instrumented recursion (7 marks)

B1.1/B1.2 - Correctness and relation to the design

The C functions follow A2's base cases and single-child halving pseudocode. The instrumentation increments a counter on entry, including the base case. Digit summation stays iterative inside recursive `digital_root`. Numeric tests deliberately stay within long long range; base 1 isolates recursion cost for large exponents without signed overflow.

Test	Actual result	What it checks
digital_root(0), (1), (9)	0, 1, 9	Single-digit/zero base case
digital_root(10)	1	First multi-digit input
digital_root(9875)	2	Repeated digit-sum recursion
digital_root(ULLONG_MAX)	6	Full 64-bit unsigned input; loop/formula agree
0^0, 0^5	1, 0	Stated zero-exponent convention and zero base
1^0, 1^30	1, 1	Identity/base cases
2^1, 2^13, 2^30	2, 8192, 1073741824	Odd/even exponents and the supplied example

Linear, halving and iterative power agree for every numerical case. `TaskB1/complete_code.c` fills only the supplied TODO region; its two output lines match the assignment exactly.

```
linear: 1073741824 in 31 calls
fast  : 1073741824 in 6 calls
```

B1.3 - Measured recursive calls

Exponent n (base=1)	Linear prediction/actual	Halving prediction/actual
10	11 / 11	5 / 5
100	101 / 101	8 / 8
1,000	1,001 / 1,001	11 / 11
10,000	10,001 / 10,001	15 / 15

Replacing the cached half product with two fresh child calls gives D(0)=1, D(n)=1+2D(floor(n/2)): for n=30 the actual count is 63, versus 6 when reusing half. Its recursion tree has six levels; total nodes are 1+2+4+8+16+32. Time becomes O(n), while maximum stack depth remains O(log n).

If the original `half=power_fast(...)` statement is retained but no longer used, the literal edit calls the child three times per non-base frame: T(n)=1+3T(floor(n/2)). Actual count is 364=1+3+9+27+81+243; time is O(n^(log2 3)). Both interpretations were executed explicitly to resolve the wording of the question.

B1.4 - Controlled recursion limit

`stack_probe.c` was compiled with -O0 -fno-optimize-sibling-calls -fno-inline. A child shell had a 1024 KiB stack and core dumps disabled. Inputs doubled from 1000 until the first failure; base 1 avoided arithmetic overflow.

```
Child stack limit: 1024 KiB; core dumps disabled; -O0 -fno-optimize-sibling-calls -fno-inline
recursive n=1000 result=1
exit_status=0
recursive n=2000 result=1
exit_status=0
recursive n=4000 result=1
exit_status=0
recursive n=8000 result=1
```

```
exit_status=0
recursive n=16000 result=1
exit_status=0
recursive n=32000 result=1
exit_status=0
tools/run_all.sh: line 35:  4607 Segmentation fault: 11  build/stack_probe recursive "$n"
exit_status=139
highest_tested_pass=32000 first_tested_failure=64000
iterative n=64000 result=1
iterative n=1000000 result=1
```

The largest tested passing exponent was 32,000; the first tested failure was 64,000 (segmentation fault, status 139). This brackets the limit for this build and stack setting; it is not the exact maximum or a universal C limit. Each recursive call consumes a frame until the bounded stack cannot grow further. The iterative version completed 64,000 and 1,000,000 with constant stack usage.

Alternative: iterative square-and-multiply

```
ALGORITHM Power_Iterative
INPUT: B and nonnegative N
OUTPUT: B raised to N
BEGIN
    result <- 1
    WHILE N > 0 DO
        IF N MOD 2 = 1 THEN result <- result * B END IF
        N <- N DIV 2
        IF N > 0 THEN B <- B * B END IF
    END WHILE
    RETURN result
END
```

Skipping the final unused square prevents unnecessary overflow at the last iteration. This is O(log n) arithmetic steps and O(1) extra space; recursive halving uses O(log n) stack, linear recursion O(n) time/stack, and a repeated-multiplication loop O(n) time/O(1) stack. All mathematical power routines require range checks or big integers if used beyond the bounded experimental inputs.

Task B2 - Adversarial input and memory checks (5 marks)

Long names: failure and correction

`long_name.txt` contains 200 X characters, followed by `Probir Saha Shohom` on a separate line. The intentionally unsafe program has `char name[16]` and unbounded `scanf("%s",name)`. AddressSanitizer reports stack-buffer-overflow, WRITE of size 201 (including the terminator), then aborts the isolated child. The actual raw diagnostic is retained in `unsafe_asan.txt`; symbolizer warnings occurred, but the overflow class, write size and stack buffer are identified. Undefined behaviour has no reliable ordinary output, so a successful-looking unsanitized run would not prove safety.

The corrected program uses fgets through `read_line`, a 64-byte buffer and a 63-character limit. It detects a missing newline, drains the rest of an oversized physical line, reports rejection, and continues from the next line. Spaces are preserved. The observed result was:

```
Rejected: donor name exceeds 63 characters; line drained.
Accepted: Probir Saha Shohom
exit_status=2 expected=2
```

`scanf("%15s",name)` could bound the original buffer, but still stops at spaces and leaves the remainder for later reads. fgets plus explicit overflow handling is better for full names.

Files, command line and malformed records

Constructed case	Actual behaviour	Interpretation
No command	Usage; exit 2	No invalid argv access
Too many find arguments	Usage; exit 2	Exact command arity enforced
Blood group X+	Usage; exit 2	Group rejected before file access
Issue quantity -2	Usage; exit 2	Unsigned quantity parser rejects signs
Missing donor file	perror: No such file or directory; exit 1	fopen failure handled
Stock file chmod 444	perror: Permission denied; exit 1	No crash or attempted use of NULL FILE pointer
Restored writable file	Issued 2 units of A-; exit 0	Valid operation succeeds
Invalid checksum / duplicate ID	Data error, original list retained	Transactional load ownership
Truncated raw binary record	Data error, original list retained	Partial final record rejected
Invalid date, numeric overflow, trailing text	Rejected by assertions in validation driver	Strict parser boundaries
INT_MAX stock + 1	Rejected, previous value retained	Prevent signed overflow

The mutation tests use copies under build/, leaving delivered input fixtures unchanged. Output includes the exact command and its expected/actual exit code. The 1000-donor valid find returned 62 O+ donors by 2026-11-18, youngest ID 1000967.

Sanitizer result and ownership limits

The corrected input reader, validation driver, registration/donation driver and 100,000-donor report were compiled with `-fsanitize=address,undefined -fno-omit-frame-pointer`. They completed without ASan/UBSan memory diagnostics. `UBSAN_OPTIONS=halt_on_error=1` made any undefined-behaviour diagnostic fatal. Donor allocation counters returned to zero, including temporary lists rejected during file loading. An extra regression checks appending a donor whose input pointer aliases the old allocation across realloc; copying it first prevents a dangling input pointer.

LeakSanitizer was disabled on this macOS run (`ASAN_OPTIONS=detect_leaks=0`), so this is not a Valgrind/LeakSanitizer proof that every possible allocation is leak-free. Explicit ownership counters cover every donor-list allocation used by these tests, and all opened files are closed. Raw logs are provided rather than a fabricated leak summary.

Alternative: getline

```
ALGORITHM Read_With_Getline
INPUT: open stream F
OUTPUT: validated donor name or error
BEGIN
    line <- NULL; capacity <- 0
    length <- GETLINE(ADDRESS(line), ADDRESS(capacity), F)
    IF length < 0 THEN FREE(line); RETURN failure END IF
    REMOVE trailing newline if present
    VALIDATE permitted name length and characters
    COPY validated name into the donor record
    FREE(line)
END
```

getline is POSIX, not ISO C11. It accepts/reallocates a heap buffer and includes the newline in the returned length. The caller owns that buffer and must free it even after an error if allocated. Reading length L costs O(L) time and O(L) heap capacity; fgets with fixed capacity K uses O(K) storage, and rejecting/draining a long line still costs O(L) time. getline avoids truncation but needs an application limit for untrusted, arbitrarily long input. The fixed Donor.name field still requires validation or a separate dynamically allocated representation.

Task B3 - Text versus binary experiments (8 marks)

B3.1 - Small-data correctness

Ten generated synthetic donors were saved and loaded in both formats. Every logical field matched: ID, name, group, age, phone, all date components and hospital. Comparison is field-by-field using `same_donor`, not memcmp over padding bytes. Both 10-record fixtures are included under data/. Text occupied 622 bytes; raw binary occupied 1040 bytes (10 x sizeof(Donor)=104). Three repetitions passed for each format and each larger size.

B3.2 - Size and timing results

The same deterministic generator creates 1000, 10,000 and 100,000 records. Each timed write includes open, write and close; each read includes parsing/validation and growing the destination list. Generation and fieldwise comparison are outside the timed interval. The C11 timespec_get clock records elapsed wall time. The table reports the median of three runs; all raw observations are in `TaskB3/measurements.csv`. Runs reuse the filesystem cache and do not call fsync, so these are application-level measurements, not guaranteed durable disk throughput.

Donors	Format	File bytes	Median write (ms)	Median read (ms)
1,000	text	61,899	0.658035	0.867844
1,000	binary	104,000	0.087023	0.506878
10,000	text	618,909	4.996061	4.621983
10,000	binary	1,040,000	0.232220	2.533913
100,000	text	6,189,009	20.752907	27.849197
100,000	binary	10,400,000	1.066923	18.640041

Text is smaller here because fixed 64-byte name storage, other fixed fields and padding are written in every binary record, while the synthetic names use fewer characters. Binary was faster in these measurements because it avoids decimal formatting/parsing, although both readers still validate each record. Both are O(n) for full write/read; observed constant factors depend on fields, compiler, storage and cache state.

B3.3 - Update the 50,000th record

The 50,000th record is zero-based index 49,999, ID 1499995. Its age was increased by one (49 to 50), leaving eligibility unchanged. Binary uses byte offset 49,999 x 104 = 5,199,896. Text loads all records, modifies the same record, writes a temporary text file and renames it. Timing is intentionally end-to-end for each strategy; opening, closing and the text rewrite are included.

Measured binary update: **0.027895 ms**. Measured text read/rewrite: **49.425125 ms**. This update was measured once per strategy; very short timings have proportionally larger noise. Every field of all 100,000 records was compared after reloading both files, verifying the changed target and unchanged neighbours/record count.

Completed stock code and invalid index

`TaskB3/complete_code.c` fills the supplied TODO with seek, one-record read, change, relative seek and one-record write. Its test output is:

```
update record 3: done
update record 9: failed
1:40 2:25 3:12 4:20 5:30
```

Index 3 selects the fourth record, group code 4. Seeking beyond the end for index 9 can succeed, but fread cannot obtain a complete record and returns zero. The function exits before fwrite, so the seek alone does not extend or alter the file; the final five printed records confirm it. The supplied exercise's main is retained, including its unchecked initial

fwrite/fclose; the reusable module adds I/O checks and bounded offsets. The exercise runs with small, valid offsets on the measured 64-bit long ABI.

B3.4 - Recommendation and fwrite alternative

Use versioned, validated text for donor exchange/auditing; it is readable and portable across endianness/ABI changes. Use binary fixed records for fast local stock updates in this exercise, but define an explicit byte-level wire format (integer widths, endianness, version, checksums) or use a transactional database for production. Raw structure layout is not a portable standard: padding can vary, and file corruption is not detected merely by a successful fread.

For an array already in memory, `fwrite(records,sizeof records[0],n,fp)` uses one library call; looping over records uses n library calls. Both transfer O(n) bytes and take O(n) total time, but batching reduces per-call overhead. Library-call count is not the same as operating-system write count because stdio buffers output. The array needs O(n) RAM; streaming one record at a time needs O(1) record RAM, while a batch of b records needs O(b). In the measured 100,000-record per-record variant, all fields matched after reload; its actual time is retained in output.txt. Because this extra run followed the earlier benchmarks, it is an illustration rather than a controlled standalone performance study.

Task B4 - Scalable nightly reporting and decision (5 marks)

B4.1 - Practical strategy

The brief gives no numerical present-day donor count, so the experiment declares a baseline of 1000 district donors and scales it by 100 to 100,000 total across 25 hospitals (4000 each). These are synthetic fixtures, not personal medical records. The report uses the same growable-list algorithm as the completed exercise, extended with name, age, phone, date and hospital fields needed by the case study.

For reporting day 2026-11-11, the next-week window is 2026-11-12 through 2026-11-18 inclusive. Include any donor who satisfies the demonstration rule on at least one day of that window, including those already eligible before it starts. The first eligible date printed is max(last donation + 90 days, window start). Age is held at its recorded value because the brief provides age rather than date of birth. Blood groups are exact labels; this program does not implement clinical compatibility rules.

```
ALGORITHM Next_Week_Report
INPUT: donor text path P, valid reporting date T, output path R
OUTPUT: next-week report and measurements
BEGIN
    L <- empty growable list
    LOAD validated donors from P into L
    start <- T + 1 day; finish <- T + 7 days
    OPEN R; WRITE column header
    FOR each donor D in L DO
        IF Eligible_On(D, finish) THEN
            eligible_date <- MAX(D.last + 90 days, start)
            WRITE D.id, group, age, hospital and eligible_date
        END IF
    END FOR
    CHECK output close; PRINT capacity, bytes and elapsed times
    FREE L.data; RESET L
END
```

Expected count follows independently from the generator: last donation is i MOD 180 days ago; eligibility by day+7 requires i MOD 180 >=83. Each full block contributes 97 records. For 100,000 records, 555 full blocks plus 100 remaining records give 555 x 97 + 17 = **53,852**. The generated report's exact ID sequence and date bounds were independently checked against that rule.

Quantity	Actual measured result
Records / capacity	100,000 / 131,072
Eligible in next-week window	53,852
sizeof(Donor)	104 bytes
Logical record payload	10,400,000 bytes (9.92 MiB)
Reserved donor array	13,631,488 bytes (13.00 MiB)
DonorList / allocator bookkeeping	40 / 24 bytes
Growth calls / potential record copies	17 / 131,070
Text load elapsed	56.667089 ms
Report scan and output elapsed	21.453142 ms
Tracked donor allocations after free	0

Reserved array bytes are capacity x sizeof(Donor), not total process RSS. Stack objects, allocator metadata, stdio buffers and executable/sanitizer overhead are not included. During a moving realloc both old and new allocations may temporarily exist. The report has O(n) scanning time, O(k) output for k matches and O(n) reserved RAM. Measurements are a single non-sanitized run; the sanitizer run is slower and is reported separately in its log.

Completed growable list and failure handling

```
added 7001 (O+): size = 1, capacity = 2
added 7002 (A+): size = 2, capacity = 2
added 7003 (B-): size = 3, capacity = 4
added 7004 (AB+): size = 4, capacity = 4
added 7005 (O-): size = 5, capacity = 8
```

`TaskB4/complete_code.c` replaces only the TODO region. A temporary p receives realloc so allocation failure cannot erase the old pointer. The supplied small demonstration main does not check add_donor's return value; it succeeded for five records here. The extended list used by the report checks all appends, caps size, guards byte multiplication and cleans partial loads. The capacity sequence confirms reuse of spare slots and doubling only when full.

B4.2 - Memory check

The full report also ran under AddressSanitizer and UndefinedBehaviorSanitizer with 100,000 records, returned zero, emitted no diagnostics and ended with zero tracked live donor allocations. Its TSV output matched the non-sanitized report byte-for-byte. Raw evidence is in `TaskB2/sanitizer_output.txt`; the ownership/leak-detection scope is explained in B2. All output streams were closed and the loaded list freed on the exercised paths.

B4.3 - Production choice and sorted index alternative

At the measured 100,000 records, a 13 MiB array and roughly 78 ms combined load/report time are affordable on this machine. Keep the growable-list implementation for this assignment and for repeated queries within a modest dataset. For a single nightly pass as the network grows, prefer streaming/batched validation and report output: it remains O(n) time but caps RAM at O(b), and does not need all donors simultaneously. A real 25-hospital service also needs coordinated unique IDs, concurrent transactions, audit/recovery and a specified clinical policy.

A separate index of fixed records `{donor_id, file_offset}` sorted by ID permits binary search:

```
ALGORITHM Lookup_Indexed_Donor
INPUT: ID, sorted index stream I of N entries, donor stream D
OUTPUT: donor or not-found
BEGIN
    low <- 0; high <- N - 1
    WHILE low <= high DO
        mid <- low + (high - low) DIV 2
        SEEK I to mid * SIZEOF(IndexEntry); READ entry
        IF entry.id = ID THEN
            SEEK D to entry.offset; READ donor; RETURN donor
        ELSE IF entry.id < ID THEN low <- mid + 1
        ELSE high <- mid - 1
        END IF
    END WHILE
    RETURN not-found
END
```

The index uses O(n) disk storage and O(1) working RAM per lookup, with O(log n) index-record reads plus one donor read. Building an index from unsorted records costs O(n log n) comparison sorting and offsets must be rebuilt/maintained when variable-length text records move. Corrupt/stale offsets require validation. Disk seeks can dominate constants.

Loading an array does **not** automatically make arbitrary donor-ID lookup O(1). An unsorted array needs O(n) search; this ID-sorted list can use O(log n) binary search. O(1) access applies to a known array index, or expected O(1) lookup needs an additional hash table/direct-address scheme and its memory/build cost. An ID index alone cannot answer a next-week eligibility query efficiently: the program still scans eligibility fields, or needs an additional index on eligible date (and maintained age/policy constraints). Thus batching suits the nightly report, while an ID index helps random donor retrieval.

Execution evidence and reproduction

Run `bash tools/run_all.sh` from the repository root. It compiles every program, verifies expected return codes, runs the three exact code exercises, regenerates datasets, performs benchmarks, probes the stack limit, and executes the corrected paths under sanitizers. The intentional unsafe program must fail; its error is expected. Repeated runs replace measurement files and may produce different timings, pointer addresses and stack failure bounds.

Each TaskA1-A5 and TaskB1-B4 folder contains its design explanation, source and execution output. Shared C modules are under common/. Small input data are under data/. Large text/binary datasets and the complete next-week TSV report are supplied in the companion Data ZIP and regenerated under build/data/. The source ZIP excludes compiled executables. All report figures and conclusions are tied to captured logs; no performance ordering is inferred from Big-O alone.

Appendix - C source listings

Each listing is labelled with its folder and filename. Related files are included in the companion ZIP and repository. Deliberate error examples are identified in their task explanations. Long lines are visually wrapped; use the original source files for execution.

TaskA1/demo.c

```
#include "../common/bank.h"
#include <assert.h>
#include <stdio.h>
#include <string.h>
int main(void){
    Allocator allocator={.fail_after=-1};DonorList list;list_init(&list,&allocator);Date
        today={11,11,2026};
    Donor a=make_donor(0,today);a.age=25;a.last=date_add(today,-100);
    Donor b=make_donor(8,today);b.age=19;b.last=date_add(today,-85);
    assert(register_donor(&list,&b)&&register_donor(&list,&a));assert(!register_donor(&list,&a));
    Eligible now=find_eligible(&list,"O+",today),next=find_eligible(&list,"O+",date_add(today,7));
    size_t count;int id;find_eligible_out(&list,"O+",today,&count,&id);
    assert(now.count==1&&now.youngest_id==a.id&&next.count==2&&next.youngest_id==b.id&&count==now.c
        ount&&id==now.youngest_id);
    printf("Today: eligible=%zu youngest_id=%d; pointer
        outputs=%zu,%d\n",now.count,now.youngest_id,count,id);
    printf("By next week: eligible=%zu youngest_id=%d\n",next.count,next.youngest_id);
    int stock[8]={10,5,8,4,7,3,6,2};assert(record_donation(&list,stock,a.id,today)&&stock[0]==11);
    assert(!record_donation(&list,stock,a.id,today)&&stock[0]==11);assert(update_stock(stock,0,-
        2)&&stock[0]==9);
    assert(!update_stock(stock,0,-10)&&stock[0]==9);
    assert(save_donors("build/demo_donors.tsv",&list)&&save_stock("build/demo_stock.bin",stock));
    DonorList loaded;list_init(&loaded,&allocator);int restored[8];

        assert(load_donors("build/demo_donors.tsv",&loaded)&&load_stock("build/demo_stock.bin",rest
        ored));
    assert(loaded.size==list.size&&same_donor(&loaded.data[0],&list.data[0])&&!memcmp(stock,restore
        d,sizeof stock));
    list_free(&loaded);list_free(&list);assert(allocator.live_blocks==0);
    puts("PASS: registration, duplicate rejection, eligibility, donation, issue and persistence
        round-trip.");
    printf("Live donor allocations after cleanup=%zu\n",allocator.live_blocks);return 0;
}
```

TaskA2/design_check.c

```
#include "bank.h"
#include <assert.h>
int main(void){
    unsigned values[]={9875,29,11,2};
    for(unsigned i=0;i<4;i++){
        unsigned result=digital_root(values[i]);assert(result==2);
        printf("digital_root(%u) = %u\n",values[i],result);
    }
    assert(digital_root(0)==0);puts("PASS: recursive trace and zero base case.");
}
```

TaskA3/pointers.c

```
#include "../common/bank.h"
#include <assert.h>
#include <stdio.h>
int main(void){
    int stock[8]={10,5,8,4,7,3,6,2};int *p=stock;
    printf("sizeof int=%zu; p+3=%p; *(p+3)=%d; p[3]=%d\n",sizeof(int),(void*)(p+3),*(p+3),p[3]);
    int previous=*p++;assert(previous==10&&p==stock+1);
    printf("*p++ returned=%d; pointer now index=%td value=%d\n",previous,p-stock,*p);
    int *minimum=lowest_stock(stock,8);assert(minimum==stock+7);*minimum+=3;
    printf("Lowest element pointer index=%td; directly updated value=%d\n",minimum-stock,*minimum);
    assert(lowest_stock(NULL,0)==NULL);puts("PASS: pointer arithmetic, post-increment and empty-
        array guard.");return 0;
}
```

TaskA4/growth.c

```
#include "../common/bank.h"
#include <assert.h>
#include <stdio.h>
static void costs(unsigned n){
    unsigned cap=0,growths=0;unsigned long long copies=0;
    for(unsigned i=0;i<n;i++)if(i==cap){copies+=i;cap=cap?2*cap:2;++growths;}
    printf("doubling,%u,%u,%u,%llu,%u\n",n,cap,growths,copies,cap-n);
    cap=0;growths=0;copies=0;
    for(unsigned i=0;i<n;i++)if(i==cap){copies+=i;cap+=10;++growths;}
    printf("plus10,%u,%u,%u,%llu,%u\n",n,cap,growths,copies,cap-n);
}
int main(void){
    Allocator allocator={.fail_after=-1};DonorList list;list_init(&list,&allocator);Date
        today={11,11,2026};
    Donor a=make_donor(0,today),b=make_donor(1,today),c=make_donor(2,today);
    assert(add_donor(&list,&a)&&add_donor(&list,&b));Donor *old=list.data;
    allocator.fail_after=(long)allocator.resize_calls;
    assert(!add_donor(&list,&c)&&list.data==old&&list.size==2&&list.capacity==2&&same_donor(&list.d
        ata[0],&a));
    puts("Injected realloc failure: old pointer, size, capacity and records preserved.");
    allocator.fail_after=-
        1;assert(add_donor(&list,&c)&&list.capacity==4);list_free(&list);list_free(&list);
    assert(allocator.live_blocks==0);puts("Cleanup is idempotent; live allocations=0.");
    puts("strategy,n,capacity,growths,logical_copy_upper_bound,unused_slots");
    costs(50);costs(50000);costs(100000);return 0;
}
```

TaskA5/storage_check.c

```
#include "bank.h"
#include <assert.h>
int main(void){
    int before[8]={10,5,8,4,7,3,6,2},after[8];
    assert(save_stock("build/a5_stock.bin",before));
    assert(issue_file("build/a5_stock.bin",3,2));
    assert(load_stock("build/a5_stock.bin",after));
    for(int i=0;i<8;i++)assert(after[i]==before[i]-(i==3?2:0));
    FILE *f=fopen("build/a5_stock.bin","rb");assert(f);assert(!fseek(f,0,SEEK_END));
    printf("8 Stock records: %ld bytes; record 3 offset: %zu bytes\n",ftell(f),3*sizeof(Stock));
    assert(!fclose(f));puts("PASS: A- units changed from 4 to 2; seven other records unchanged.");
}
```

TaskB1/complete_code.c

```
#include <stdio.h>
long long calls = 0;     /* number of recursive calls made */
/* Linear recursion: base^exp = base * base^(exp-1) */
long long power_linear(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}
/* Divide-and-conquer recursion: compute base^(exp/2) once and square it. */
long long power_fast(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;
    long long half = power_fast(base, exp / 2);
    return exp % 2 ? half * half * base : half * half;
}
int main(void)
{
    calls = 0;
    long long a = power_linear(2, 30);
    printf("linear: %lld in %lld calls\n", a, calls);
    calls = 0;
    long long b = power_fast(2, 30);
    printf("fast  : %lld in %lld calls\n", b, calls);
    return 0;
}
```

TaskB1/recursion.c

```
#include "bank.h"
#include <assert.h>
#include <limits.h>
static unsigned long long calls;
/* Preconditions: exponent nonnegative and every multiplication fits long long. */
static long long linear(long long b,unsigned n){++calls;return n?b*linear(b,n-1):1;}
static long long fast(long long b,unsigned n){++calls;if(!n)return 1;long long h=fast(b,n/2);return
     n%2?h*h*b:h*h;}
static long long iterative(long long b,unsigned n){
    long long result=1;
    while(n){if(n%2)result*=b;n/=2;if(n)b*=b;}
    return result;
}
static long long duplicated(long long b,unsigned n){
    ++calls;if(!n)return 1;
    long long a=duplicated(b,n/2),c=duplicated(b,n/2);
    return n%2?a*c*b:a*c;
}
static long long triplicated(long long b,unsigned n){
    ++calls;if(!n)return 1;
    long long unused=triplicated(b,n/2);(void)unused;
    long long a=triplicated(b,n/2),c=triplicated(b,n/2);
    return n%2?a*c*b:a*c;
}
static unsigned root_loop(unsigned long long n){while(n>=10){unsigned
    s=0;while(n){s+=(unsigned)(n%10);n/=10;}n=s;}return (unsigned)n;}
int main(void){
    unsigned long long roots[]={0,1,9,10,9875,ULLONG_MAX};
    unsigned expected[]={0,1,9,1,2,6};
    for(size_t i=0;i<sizeof roots/sizeof roots[0];i++){
        unsigned recursive=digital_root(roots[i]),loop=root_loop(roots[i]);
        unsigned formula=roots[i]?1+(unsigned)((roots[i]-1)%9):0;
        assert(recursive==expected[i]&&loop==recursive&&formula==recursive);
        printf("digital_root(%llu) = %u; loop/formula agree\n",roots[i],recursive);
    }
    struct {long long b;unsigned n;long long want;}
        tests[]={{0,0,1},{0,5,0},{1,0,1},{1,30,1},{2,1,2},{2,13,8192},{2,30,1073741824}};
    for(size_t i=0;i<sizeof tests/sizeof tests[0];i++){
        long long
            a=linear(tests[i].b,tests[i].n),b=fast(tests[i].b,tests[i].n),c=iterative(tests[i].b,te
            sts[i].n);
        assert(a==tests[i].want&&b==a&&c==a);
        printf("%lld^%u = %lld; all three versions agree\n",tests[i].b,tests[i].n,a);
    }
    puts("n,linear_calls,fast_calls");unsigned ns[]={10,100,1000,10000};
    for(size_t i=0;i<4;i++){calls=0;assert(linear(1,ns[i])==1);unsigned long long
        a=calls;calls=0;assert(fast(1,ns[i])==1);printf("%u,%llu,%llu\n",ns[i],a,calls);}
    calls=0;assert(duplicated(2,30)==1073741824);printf("two recursive half calls: %llu
        calls\n",calls);assert(calls==63);
    calls=0;assert(triplicated(2,30)==1073741824);printf("retaining unused half call: %llu
        calls\n",calls);assert(calls==364);
    puts("PASS: bounded arithmetic inputs; 0^0 is defined as 1 for this exercise.");
}
```

TaskB1/stack_probe.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* Compile at -O0 with sibling-call optimisation disabled. */
static unsigned recurse(unsigned n){volatile unsigned x=n;if(!x)return 1;return recurse(x-1)*1U;}
int main(int argc,char **argv){
    if(argc!=3)return 2;char *end;unsigned long n=strtoul(argv[2],&end,10);
    if(*end||n>1000000)return 2;
    unsigned result=1;
    if(!strcmp(argv[1],"recursive"))result=recurse((unsigned)n);
    else if(!strcmp(argv[1],"iterative")){for(unsigned long i=0;i<n;i++)result*=1U;}
    else return 2;
    printf("%s n=%lu result=%u\n",argv[1],n,result);return result!=1;
}
```

TaskB2/main.c

```
#include "bank.h"
#include <string.h>
#include <limits.h>
static int usage(void){
    fputs("Usage:\n  bloodbank find DONORS.tsv GROUP YYYY-MM-DD\n  bloodbank issue STOCK.bin GROUP
        UNITS\n  bloodbank stock STOCK.bin\n",stderr);return 2;
}
int main(int argc,char **argv){
    if(argc<2)return usage();
    if(!strcmp(argv[1],"find")){
        Date date;if(argc!=5||group_index(argv[3])<0||!parse_date(argv[4],&date))return usage();
        Allocator a={.fail_after=-1};DonorList list;list_init(&list,&a);
        if(!load_donors(argv[2],&list)){list_free(&list);return 1;}
        Eligible e=find_eligible(&list,argv[3],date);
        printf("Eligible %s donors: %zu; youngest ID: %d\n",argv[3],e.count,e.youngest_id);
        list_free(&list);return a.live_blocks!=0;
    }
    if(!strcmp(argv[1],"issue")){
        unsigned quantity;if(argc!=5||group_index(argv[3])<0||!parse_unsigned(argv[4],INT_MAX,&quan
            tity)||!quantity)return usage();
        if(!issue_file(argv[2],group_index(argv[3]),quantity))return 1;
        printf("Issued %u units of %s\n",quantity,argv[3]);return 0;
    }
    if(!strcmp(argv[1],"stock")){
        if(argc!=3)return usage();int stock[GROUP_COUNT];if(!load_stock(argv[2],stock))return 1;
        for(int i=0;i<GROUP_COUNT;i++)printf("%s: %d\n",GROUPS[i],stock[i]);return 0;
    }
    return usage();
}
```

TaskB2/safe_name.c

```
#include "bank.h"
int main(void){
    char name[64];int status,bad=0;
    while((status=read_line(stdin,name,sizeof name))!=0){
        if(status==-2){puts("Rejected: donor name exceeds 63 characters; line
            drained.");bad=1;continue;}
        if(status<0){perror("read name");return 1;}
        printf("Accepted: %s\n",name);
    }
    return bad?2:0;
}
```

TaskB2/unsafe_name.c

```
/* Intentionally unsafe assignment experiment; run only in a child with ASan. */
#include <stdio.h>
int main(void){char name[16];if(scanf("%s",name)!=1)return 1;printf("Name: %s\n",name);return 0;}
```

TaskB2/validation.c

```
#include "bank.h"
#include <assert.h>
#include <string.h>
#include <limits.h>
int main(void){
    Allocator a={.fail_after=-1};DonorList list;list_init(&list,&a);
    Donor d=make_donor(0,(Date){11,11,2026});assert(add_donor(&list,&d));
    FILE *f=fopen("build/bad.tsv","w");assert(f);fputs("DONORS\t1\n1000002\tBad
        Check\tO+\t25\t0000000000\t2026-01-01\t1\n",f);assert(!fclose(f));
    assert(!load_donors("build/bad.tsv",&list));assert(list.size==1&&same_donor(&d,&list.data[0]));
    f=fopen("build/bad.bin","wb");assert(f);assert(fwrite(&d,1,sizeof d-1,f)==sizeof
        d-1);assert(!fclose(f));
    assert(!load_binary_donors("build/bad.bin",&list));assert(list.size==1&&same_donor(&d,&list.dat
        a[0]));
    f=fopen("build/duplicate.tsv","w");assert(f);fputs("DONORS\t1\n",f);
    for(int
        i=0;i<2;i++)fprintf(f,"%d\t%s\t%s\t%u\t%s\t2026-11-
        11\t1\n",d.id,d.name,d.group,d.age,d.phone);
    assert(!fclose(f));assert(!load_donors("build/duplicate.tsv",&list));assert(list.size==1);
    Date date;unsigned
        n;assert(!parse_date("2026-02-29",&date));assert(parse_date("2024-02-29",&date));
    assert(!parse_unsigned("-
        2",INT_MAX,&n)&&!parse_unsigned("2x",INT_MAX,&n)&&!parse_unsigned("999999999999999999",I
        NT_MAX,&n));
    assert(group_index(NULL)==-1); /* NULL maps to -1. */
    assert(group_index("X+")==-1);
    Donor invalid=d;invalid.id++;assert(!register_donor(&list,&invalid));
    int
        stock[GROUP_COUNT]={0};stock[0]=INT_MAX;assert(!update_stock(stock,0,1)&&stock[0]==INT_MAX)
        ;
    /* Alias-safe append through a realloc and cleanup of all owned buffers. */
    assert(add_donor(&list,&d));assert(add_donor(&list,&list.data[0]));assert(same_donor(&list.data
        [0],&list.data[2]));
    list_free(&list);assert(a.live_blocks==0);list_free(&list);
    printf("PASS: malformed checksum, duplicate ID, truncated binary, invalid date/numbers,
        overflow and alias-safe resize\nTracked live allocations: %zu\n",a.live_blocks);
}
```

TaskB3/benchmark.c

```
#include "bank.h"
#include <assert.h>
#include <time.h>
#include <string.h>
static double now(void){struct timespec t;assert(timespec_get(&t,TIME_UTC)==TIME_UTC);return
    (double)t.tv_sec+t.tv_nsec/1e9;}
static long length(const char *path){FILE
    *f=fopen(path,"rb");assert(f);assert(!fseek(f,0,SEEK_END));long
    n=ftell(f);assert(n>=0&&!fclose(f));return n;}
static void equal(const DonorList *a,const DonorList *b){assert(a->size==b->size);for(int
    i=0;i<a->size;i++)assert(same_donor(&a->data[i],&b->data[i]));}
int main(void){
    Allocator a={.fail_after=-1};DonorList
        source,loaded;list_init(&source,&a);list_init(&loaded,&a);
    unsigned sizes[]={10,1000,10000,100000};Date today={11,11,2026};
    printf("sizeof_Donor=%zu; sizeof_Stock=%zu\n",sizeof(Donor),sizeof(Stock));
    FILE *csv=fopen("TaskB3/measurements.csv","w");assert(csv);fputs("n,run,format,bytes,write_ms,r
        ead_ms\n",csv);
    for(size_t k=0;k<4;k++){
        unsigned n=sizes[k];while((unsigned)source.size<n){Donor
            d=make_donor((unsigned)source.size,today);assert(add_donor(&source,&d));}
        char txt[80],bin[80];snprintf(txt,sizeof
            txt,"build/data/donors_%u.tsv",n);snprintf(bin,sizeof
            bin,"build/data/donors_%u.bin",n);
        for(int run=1;run<=3;run++){
            double start=now();assert(save_donors(txt,&source));double wt=(now()-start)*1000;
            start=now();assert(load_donors(txt,&loaded));double
                rt=(now()-start)*1000;equal(&source,&loaded);list_free(&loaded);
            fprintf(csv,"%u,%d,text,%ld,%.6f,%.6f\n",n,run,length(txt),wt,rt);
            start=now();assert(save_binary_donors(bin,&source));wt=(now()-start)*1000;
            start=now();assert(load_binary_donors(bin,&loaded));rt=(now()-
                start)*1000;equal(&source,&loaded);list_free(&loaded);
            fprintf(csv,"%u,%d,binary,%ld,%.6f,%.6f\n",n,run,length(bin),wt,rt);
        }
        printf("n=%u: all text/binary round trips match every field; bytes text=%ld
            binary=%ld\n",n,length(txt),length(bin));
    }
    assert(!fclose(csv));
    /* Update the 50,000th record (zero-based index 49,999), then check EVERY record. */
    source.data[49999].age++;Donor changed=source.data[49999];
    double
        start=now();assert(update_binary_donor("build/data/donors_100000.bin",49999,&changed));doub
        le binary_ms=(now()-start)*1000;

        start=now();assert(load_donors("build/data/donors_100000.tsv",&loaded));loaded.data[49999]=
        changed;
    assert(save_donors("build/data/donors_updated.tmp",&loaded));assert(!rename("build/data/donors_
        updated.tmp","build/data/donors_100000.tsv"));double text_ms=(now()-start)*1000;
    list_free(&loaded);assert(load_donors("build/data/donors_100000.tsv",&loaded));equal(&source,&l
        oaded);list_free(&loaded);
    assert(load_binary_donors("build/data/donors_100000.bin",&loaded));equal(&source,&loaded);list_
        free(&loaded);
    csv=fopen("TaskB3/update_measurements.csv","w");assert(csv);fprintf(csv,"index,id,binary_ms,tex
        t_read_rewrite_ms\n49999,%d,%.6f,%.6f\n",changed.id,binary_ms,text_ms);assert(!fclose(csv))
        ;
    printf("50,000th donor ID=%d: binary %.6f ms; text read/rewrite %.6f ms; all 100000 records
        verified\n",changed.id,binary_ms,text_ms);
    /* Compare one fwrite(array) against n fwrite(record) calls. */
    FILE *f=fopen("build/data/per_record.bin","wb");assert(f);start=now();
    for(int i=0;i<source.size;i++)assert(fwrite(&source.data[i],sizeof(Donor),1,f)==1);
    assert(!fclose(f));printf("Per-record fwrite: %d library calls, %.6f ms; batch API uses 1
        call\n",source.size,(now()-start)*1000);
    assert(load_binary_donors("build/data/per_record.bin",&loaded));equal(&source,&loaded);list_fre
        e(&loaded);
    list_free(&source);assert(a.live_blocks==0);printf("Tracked live allocations:
        %zu\n",a.live_blocks);
}
```

TaskB3/complete_code.c

```
#include <stdio.h>
typedef struct { int group_code; int units; } Stock;   /* fixed-size binary record */
/* Overwrites the units of record number index directly in the binary file. */
int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fp == NULL || index < 0 || units < 0) return 0;
    if (fseek(fp, (long)index * (long)sizeof s, SEEK_SET) != 0) return 0;
    if (fread(&s, sizeof s, 1, fp) != 1) return 0;
    s.units = units;
    if (fseek(fp, -(long)sizeof s, SEEK_CUR) != 0) return 0;
    return fwrite(&s, sizeof s, 1, fp) == 1;
}
int main(void)
{
    FILE *fp = tmpfile();
    if (fp == NULL) { perror("tmpfile"); return 1; }
    Stock initial[5] = {{1, 40}, {2, 25}, {3, 12}, {4, 8}, {5, 30}};
    fwrite(initial, sizeof(Stock), 5, fp);
    printf("update record 3: %s\n", update_units(fp, 3, 20) ? "done" : "failed");
    printf("update record 9: %s\n", update_units(fp, 9, 20) ? "done" : "failed");
    rewind(fp);
    Stock s;
    while (fread(&s, sizeof s, 1, fp) == 1) printf("%d:%d ", s.group_code, s.units);
    printf("\n");
    fclose(fp);
    return 0;
}
```

TaskB4/complete_code.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;
/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group)
{
    if (list->size == list->capacity) {
        int new_capacity = list->capacity == 0 ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)new_capacity * sizeof *p);
        if (p == NULL) return 0;
        list->data = p;
        list->capacity = new_capacity;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->size++;
    return 1;
}
int main(void)
{
    DonorList list = {NULL, 0, 0};
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};
    for (int i = 0; i < 5; i++) {
        add_donor(&list, 7001 + i, groups[i]);
        printf("added %d (%s): size = %d, capacity = %d\n",
               list.data[i].id, list.data[i].group, list.size, list.capacity);
    }
    free(list.data);
    return 0;
}
```

TaskB4/report.c

```
#include "bank.h"
#include <assert.h>
#include <time.h>
static double now(void){struct timespec t;assert(timespec_get(&t,TIME_UTC)==TIME_UTC);return
    (double)t.tv_sec+t.tv_nsec/1e9;}
int main(int argc,char **argv){
    Date today;if(argc!=4||!parse_date(argv[2],&today)||date_ordinal(today)>date_ordinal((Date){24,
        12,9999})){
        fputs("Usage: report DONORS.tsv YYYY-MM-DD REPORT.tsv\n",stderr);return 2;
    }
    Allocator a={.fail_after=-1};DonorList list;list_init(&list,&a);
    double start=now();if(!load_donors(argv[1],&list))return 1;double load_ms=(now()-start)*1000;
    FILE *f=fopen(argv[3],"w");if(!f){perror("open report");list_free(&list);return 1;}
    Date first=date_add(today,1),last=date_add(today,7);size_t count=0,hospitals[25]={0};
    start=now();int ok=fprintf(f,"id\tgroup\tage\thospital\tfirst_eligible_in_window\n")>=0;
    for(int i=0;ok&&i<list.size;i++){
        const Donor *d=&list.data[i];++hospitals[d->hospital-1];
        if(!eligible_on(d,last))continue;
        Date
            eligible=date_add(d-
            >last,90);if(date_ordinal(eligible)<date_ordinal(first))eligible=first;
        ok=fprintf(f,"%d\t%s\t%u\t%u\t%04d-%02d-%02d\n",d->id,d->group,d->age,d-
            >hospital,eligible.year,eligible.month,eligible.day)>=0;++count;
    }
    if(fclose(f))ok=0;double report_ms=(now()-start)*1000;
    if(!ok){perror("write report");list_free(&list);return 1;}
    printf("Next-week window: %04d-%02d-%02d through %04d-%02d-%02d
        (inclusive)\n",first.year,first.month,first.day,last.year,last.month,last.day);
    printf("Donors=%d capacity=%d eligible=%zu\n",list.size,list.capacity,count);
    printf("sizeof(Donor)=%zu payload_bytes=%zu reserved_bytes=%zu list_bytes=%zu allocator_bytes=%
        zu\n",sizeof(Donor),(size_t)list.size*sizeof(Donor),(size_t)list.capacity*sizeof(Donor),siz
        eof list,sizeof a);
    printf("Growth calls=%zu logical_copy_bound=%zu; load_ms=%.6f
        report_ms=%.6f\n",list.growths,list.copy_bound,load_ms,report_ms);
    printf("Hospital counts:");for(int i=0;i<25;i++)printf(" %d:%zu",i+1,hospitals[i]);puts("");
    list_free(&list);assert(a.live_blocks==0);printf("Tracked live allocations after cleanup:
        %zu\n",a.live_blocks);return 0;
}
```

common/bank.c

```
#include "bank.h"
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <stdint.h>
#include <errno.h>
const char *const GROUPS[GROUP_COUNT]={"O+","O-","A+","A-","B+","B-","AB+","AB-"};
int group_index(const char *group){if(!group)return -1;for(int
    i=0;i<GROUP_COUNT;i++)if(!strcmp(group,GROUPS[i]))return i;return -1;}
unsigned digital_root(unsigned long long n){if(n<10)return (unsigned)n;unsigned
    sum=0;while(n){sum+=(unsigned)(n%10);n/=10;}return digital_root(sum);}
bool parse_unsigned(const char *s,unsigned max,unsigned *out){
    if(!s||!*s||!out)return false;
    for(const char *p=s;*p;p++)if(*p<'0'||*p>'9')return false;
    errno=0;char *end;unsigned long v=strtoul(s,&end,10);
    if(errno==ERANGE||*end||v>max)return false;*out=(unsigned)v;return true;
}
bool parse_date(const char *s,Date *out){
    if(!s||!out||strlen(s)!=10||s[4]!='-'||s[7]!='-')return false;
    for(int i=0;i<10;i++)if(i!=4&&i!=7&&(s[i]<'0'||s[i]>'9'))return false;
    Date d={(s[8]-'0')*10+s[9]-'0',(s[5]-'0')*10+s[6]-'0',
        (s[0]-'0')*1000+(s[1]-'0')*100+(s[2]-'0')*10+s[3]-'0'};
    if(!date_valid(d))return false;*out=d;return true;
}
Date date_from_ordinal(int n){
    int low=1900,high=9999;
    while(low<high){int mid=low+(high-low+1)/2;if(date_ordinal((Date){1,1,mid})<=n)low=mid;else
        high=mid-1;}
    int month=1;while(month<12&&date_ordinal((Date){1,month+1,low})<=n)++month;
    return (Date){n-date_ordinal((Date){1,month,low})+1,month,low};
}
Date date_add(Date d,int days){return date_from_ordinal(date_ordinal(d)+days);}
bool donor_valid(const Donor *d){
    if(!d||d->id<10||d->id%10!=(int)digital_root((unsigned)(d->id/10))||
       !memchr(d->name,0,sizeof d->name)||!d->name[0]||!memchr(d->group,0,sizeof d->group)||
       group_index(d->group)<0||d->age>130||!memchr(d->phone,0,sizeof d->phone)||
       strlen(d->phone)!=10||!date_valid(d->last)||d->hospital<1||d->hospital>25)return false;
    for(const unsigned char *p=(const unsigned char*)d->name;*p;p++)if(*p<32||*p==127)return false;
    for(int i=0;i<10;i++)if(d->phone[i]<'0'||d->phone[i]>'9')return false;
    return true;
}
void list_init(DonorList *list,Allocator *a){*list=(DonorList){.allocator=a};}
void list_free(DonorList *list){
    if(list->data){free(list->data);if(list->allocator)--list->allocator->live_blocks;}
    Allocator *a=list->allocator;*list=(DonorList){.allocator=a};
}
int add_donor(DonorList *list,const Donor *d){
    if(!list||!donor_valid(d)||list->size>=MAX_DONORS)return 0;
    Donor copy=*d; /* Preserve a donor that aliases the old allocation. */
    if(list->size==list->capacity){
        int capacity=list->capacity?list->capacity*2:2;
        if(capacity>MAX_DONORS)capacity=MAX_DONORS;
        if((size_t)capacity>SIZE_MAX/sizeof(Donor))return 0;
        Allocator *a=list->allocator;
        if(a){if(a->fail_after>=0&&(long)a->resize_calls>=a->fail_after)return
            0;++a->resize_calls;}
        bool first=list->data==NULL;
        Donor *p=realloc(list->data,(size_t)capacity*sizeof *p);
        if(!p)return 0;
        if(a&&first)++a->live_blocks;
        list->copy_bound+=(size_t)list->size;++list->growths;
        list->data=p;list->capacity=capacity;
    }
    list->data[list->size++]=copy;return 1;
}
int register_donor(DonorList *list,const Donor *d){
    if(!list||!donor_valid(d))return 0;
    int low=0,high=list->size;
    while(low<high){int mid=low+(high-low)/2;if(list->data[mid].id<d->id)low=mid+1;else high=mid;}
    if(low<list->size&&list->data[low].id==d->id)return 0;
    Donor copy=*d;int previous_size=list->size;
    if(!add_donor(list,&copy))return 0;
    memmove(&list->data[low+1],&list->data[low],(size_t)(previous_size-low)*sizeof copy);
    list->data[low]=copy;return 1;
}
bool eligible_on(const Donor *d,Date on){
    /* Demonstration policy only: age18..65 and90days since recorded donation. */
    return date_valid(on)&&d->age>=18&&d->age<=65&&date_ordinal(on)-date_ordinal(d->last)>=90;
```

```
}
Eligible find_eligible(const DonorList *list,const char *group,Date on){
    Eligible result={0,0,0};
    for(int i=0;i<list->size;i++){
        const Donor *d=&list->data[i];
        if(strcmp(d->group,group)||!eligible_on(d,on))continue;
        if(!result.count||d->age<result.youngest_age||(d->age==result.youngest_age&&d-
            >id<result.youngest_id)){
            result.youngest_id=d->id;result.youngest_age=d->age;
        }
        ++result.count;
    }
    return result;
}
void find_eligible_out(const DonorList *list,const char *group,Date on,size_t *count,int *id){
    Eligible e=find_eligible(list,group,on);if(count)*count=e.count;if(id)*id=e.youngest_id;
}
int *lowest_stock(int stock[],size_t n){if(!stock||!n)return NULL;int *best=stock;for(size_t
    i=1;i<n;i++)if(stock[i]<*best)best=&stock[i];return best;}
int update_stock(int stock[GROUP_COUNT],int group,int delta){
    if(!stock||group<0||group>=GROUP_COUNT||stock[group]<0)return 0;
    long long next=(long long)stock[group]+delta;if(next<0||next>INT_MAX)return 0;
    stock[group]=(int)next;return 1;
}
int record_donation(DonorList *list,int stock[GROUP_COUNT],int id,Date on){
    if(!list||!stock||!date_valid(on))return 0;
    for(int i=0;i<list->size;i++)if(list->data[i].id==id){
        Donor
            *d=&list->data[i];if(!eligible_on(d,on)||!update_stock(stock,group_index(d-
            >group),1))return 0;
        d->last=on;return 1;
    }
    return 0;
}
bool same_donor(const Donor *a,const Donor *b){
    return a->id==b->id&&!strcmp(a->name,b->name)&&!strcmp(a->group,b->group)&&a->age==b->age&&
           !strcmp(a->phone,b->phone)&&a->last.day==b->last.day&&a->last.month==b->last.month&&a-
               >last.year==b->last.year&&a->hospital==b->hospital;
}
Donor make_donor(unsigned i,Date today){
    Donor d={0};unsigned body=100000+i;
    d.id=(int)(body*10+digital_root(body));snprintf(d.name,sizeof d.name,"Synthetic Donor %06u",i);
    strcpy(d.group,GROUPS[i%8]);d.age=18+i%48;snprintf(d.phone,sizeof d.phone,"%010u",i);
    d.last=date_add(today,-(int)(i%180));d.hospital=1+i%25;return d;
}
```

common/bank.h

```
#ifndef CSA02_BANK_H
#define CSA02_BANK_H
#include <stdio.h>
#include <stdbool.h>
#include <stddef.h>
#include "date.h"
#define GROUP_COUNT 8
#define MAX_DONORS 1000000
typedef struct {int id;char name[64],group[4];unsigned age;char phone[11];Date last;unsigned
    hospital;} Donor;
typedef struct {size_t live_blocks,resize_calls;long fail_after;} Allocator;
typedef struct {Donor *data;int size,capacity;size_t growths,copy_bound;Allocator *allocator;}
    DonorList;
typedef struct {int group_code,units;} Stock;
typedef struct {size_t count;int youngest_id;unsigned youngest_age;} Eligible;
extern const char *const GROUPS[GROUP_COUNT];
int group_index(const char *group);
unsigned digital_root(unsigned long long value);
bool donor_valid(const Donor *donor);
void list_init(DonorList *list,Allocator *allocator);
void list_free(DonorList *list);
int add_donor(DonorList *list,const Donor *donor);
int register_donor(DonorList *list,const Donor *donor);
bool eligible_on(const Donor *donor,Date on);
Eligible find_eligible(const DonorList *list,const char *group,Date on);
void find_eligible_out(const DonorList *list,const char *group,Date on,size_t *count,int
    *youngest_id);
int *lowest_stock(int stock[],size_t count);
int update_stock(int stock[GROUP_COUNT],int group,int delta);
int record_donation(DonorList *list,int stock[GROUP_COUNT],int id,Date on);
bool parse_date(const char *text,Date *out);
bool parse_unsigned(const char *text,unsigned maximum,unsigned *out);
Date date_from_ordinal(int ordinal);
Date date_add(Date date,int days);
int read_line(FILE *file,char *buffer,size_t capacity); /* 1 line,0 EOF,-1 I/O,-2 long */
int save_donors(const char *path,const DonorList *list);
int load_donors(const char *path,DonorList *list);
int save_binary_donors(const char *path,const DonorList *list);
int load_binary_donors(const char *path,DonorList *list);
int save_stock(const char *path,const int stock[GROUP_COUNT]);
int load_stock(const char *path,int stock[GROUP_COUNT]);
int issue_file(const char *path,int group,unsigned units);
int update_binary_donor(const char *path,size_t index,const Donor *donor);
bool same_donor(const Donor *a,const Donor *b);
Donor make_donor(unsigned sequence,Date today);
#endif
```

common/date.c

```
#include "date.h"
static bool leap(int y){return y%4==0&&(y%100!=0||y%400==0);}
bool date_valid(Date d){
    const int days[]={31,28,31,30,31,30,31,31,30,31,30,31};
    if(d.year<1900||d.year>9999||d.month<1||d.month>12)return false;
    return d.day>=1&&d.day<=days[d.month-1]+(d.month==2&&leap(d.year));
}
int date_ordinal(Date d){
    const int before[]={0,31,59,90,120,151,181,212,243,273,304,334};
    int y=d.year-1;
    return 365*y+y/4-y/100+y/400+before[d.month-1]+(d.month>2&&leap(d.year))+d.day;
}
```

common/date.h

```
#ifndef CSA02_DATE_H
#define CSA02_DATE_H
#include <stdbool.h>
typedef struct { int day,month,year; } Date;
bool date_valid(Date d);
int date_ordinal(Date d); /* valid dates only; Gregorian day number */
#endif
```

common/storage.c

```
#include "bank.h"
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>
int read_line(FILE *f,char *s,size_t cap){
    if(!f||!s||cap<2||cap>INT_MAX)return -1;
    if(!fgets(s,(int)cap,f))return ferror(f)?-1:0;
    char *newline=strchr(s,'\n');if(newline){*newline=0;return 1;}
    int c=fgetc(f);if(c=='\n'||c==EOF)return ferror(f)?-1:1;
    while((c=fgetc(f))!='\n'&&c!=EOF){}return -2;
}
static int invalid(const char *context){errno=EINVAL;perror(context);return 0;}
int save_donors(const char *path,const DonorList *list){
    FILE *f=fopen(path,"w");if(!f){perror("open donor text for writing");return 0;}
    int ok=fprintf(f,"DONORS\t1\n")>=0;
    for(int i=0;ok&&i<list->size;i++){
        const Donor *d=&list->data[i];
        ok=fprintf(f,"%d\t%s\t%s\t%u\t%s\t%04d-%02d-%02d\t%u\n",d->id,d->name,d->group,d->age,d-
            >phone,d->last.year,d->last.month,d->last.day,d->hospital)>=0;
    }
    if(fclose(f)!=0)ok=0;if(!ok)perror("write donor text");return ok;
}
static bool parse_donor(char *line,Donor *d){
    char *fields[7];fields[0]=line;unsigned count=1;
    for(char *p=line;*p;p++)if(*p=='\t'){if(count==7)return false;*p=0;fields[count++]=p+1;}
    if(count!=7)return false;
    unsigned id,age,hospital;Date last;
    if(!parse_unsigned(fields[0],INT_MAX,&id)||!parse_unsigned(fields[3],130,&age)||
       !parse_unsigned(fields[6],25,&hospital)||strlen(fields[1])>=sizeof d->name||
       strlen(fields[2])>=sizeof d->group||strlen(fields[4])>=sizeof
           d->phone||!parse_date(fields[5],&last))return false;
    *d=(Donor){.id=(int)id,.age=age,.last=last,.hospital=hospital};
    strcpy(d->name,fields[1]);strcpy(d->group,fields[2]);strcpy(d->phone,fields[4]);return
        donor_valid(d);
}
int load_donors(const char *path,DonorList *list){
    FILE *f=fopen(path,"r");if(!f){perror("open donor text");return 0;}
    DonorList temporary;list_init(&temporary,list->allocator);char line[256];int
        status=read_line(f,line,sizeof line);
    int ok=status==1&&!strcmp(line,"DONORS\t1");
    while(ok&&(status=read_line(f,line,sizeof line))==1){
        Donor d;if(!parse_donor(line,&d)||(temporary.size&&temporary.data[temporary.size-
            1].id>=d.id)||!add_donor(&temporary,&d)){ok=0;break;}
    }
    if(status<0||ferror(f))ok=0;if(fclose(f)!=0)ok=0;
    if(!ok){list_free(&temporary);return invalid("load donor text: malformed, unordered or
        allocation failure");}
    list_free(list);*list=temporary;return 1;
}
int save_binary_donors(const char *path,const DonorList *list){
    FILE *f=fopen(path,"wb");if(!f){perror("open donor binary for writing");return 0;}
    int ok=fwrite(list->data,sizeof(Donor),(size_t)list->size,f)==(size_t)list->size;
    if(fclose(f)!=0)ok=0;if(!ok)perror("write donor binary");return ok;
}
int load_binary_donors(const char *path,DonorList *list){
    FILE *f=fopen(path,"rb");if(!f){perror("open donor binary");return 0;}
    DonorList temporary;list_init(&temporary,list->allocator);Donor d;size_t bytes;int ok=1;
    while((bytes=fread(&d,1,sizeof d,f))!=0){
        if(bytes!=sizeof d||!donor_valid(&d)||(temporary.size&&temporary.data[temporary.size-
            1].id>=d.id)||!add_donor(&temporary,&d)){ok=0;break;}
    }
    if(ferror(f))ok=0;if(fclose(f)!=0)ok=0;
    if(!ok){list_free(&temporary);return invalid("load donor binary");}
    list_free(list);*list=temporary;return 1;
}
int save_stock(const char *path,const int stock[GROUP_COUNT]){
    Stock records[GROUP_COUNT];for(int i=0;i<GROUP_COUNT;i++){if(stock[i]<0)return
        invalid("negative stock");records[i]=(Stock){i,stock[i]};}
    FILE *f=fopen(path,"wb");if(!f){perror("open stock for writing");return 0;}
    int ok=fwrite(records,sizeof records[0],GROUP_COUNT,f)==GROUP_COUNT;
    if(fclose(f)!=0)ok=0;if(!ok)perror("write stock");return ok;
}
int load_stock(const char *path,int stock[GROUP_COUNT]){
    FILE *f=fopen(path,"rb");if(!f){perror("open stock");return 0;}
    Stock r[GROUP_COUNT];int ok=fread(r,sizeof r[0],GROUP_COUNT,f)==GROUP_COUNT;
    if(ok&&(fgetc(f)!=EOF||ferror(f)))ok=0;
    for(int i=0;ok&&i<GROUP_COUNT;i++)if(r[i].group_code!=i||r[i].units<0)ok=0;
```

```
    if(fclose(f)!=0)ok=0;if(!ok)return invalid("invalid stock file");
    for(int i=0;i<GROUP_COUNT;i++)stock[i]=r[i].units;return 1;
}
int issue_file(const char *path,int group,unsigned quantity){
    if(group<0||group>=GROUP_COUNT||!quantity||quantity>INT_MAX)return invalid("invalid issue
        request");
    FILE *f=fopen(path,"r+b");if(!f){perror("open stock for update");return 0;}
    Stock r;long offset=(long)group*(long)sizeof r;int ok=1;
    if(fseek(f,offset,SEEK_SET)||fread(&r,sizeof
        r,1,f)!=1||r.group_code!=group||r.units<0||quantity>(unsigned)r.units)ok=0;
    if(ok){r.units-=(int)quantity;if(fseek(f,offset,SEEK_SET)||fwrite(&r,sizeof
        r,1,f)!=1||fflush(f))ok=0;}
    if(fclose(f)!=0)ok=0;if(!ok)return invalid("issue failed: invalid/insufficient stock or
        I/O");return 1;
}
int update_binary_donor(const char *path,size_t index,const Donor *d){
    if(!donor_valid(d)||index>(size_t)LONG_MAX/sizeof *d)return invalid("invalid donor update");
    FILE *f=fopen(path,"r+b");if(!f){perror("open donor binary for update");return 0;}
    Donor old;long offset=(long)(index*sizeof *d);int ok=1;
    if(fseek(f,offset,SEEK_SET)||fread(&old,sizeof old,1,f)!=1||old.id!=d->id)ok=0;
    if(ok&&(fseek(f,offset,SEEK_SET)||fwrite(d,sizeof *d,1,f)!=1||fflush(f)))ok=0;
    if(fclose(f)!=0)ok=0;if(!ok)return invalid("binary donor update failed");return 1;
}
```